



PRODUCTION TECHNICAL SPECIFICATIONS & FEATURE TICKETS

Project: Sides of Mashriq - Premium Fusion Dining Engine

Stack: MERN (MongoDB Atlas M0, Express.js, React, Node.js) with Socket.io

Version: 1.0.0

1. Directory Structure Blueprint (Monorepo Setup)

To keep development frictionless during our 7-day sprint, we configure a clean, single-repository structure containing a separated frontend and backend.

```
sides-of-mashriq/
├── server/                # Node.js + Express Backend
│   ├── config/           # DB & Socket configurations
│   ├── controllers/      # Express request handlers
│   ├── middleware/       # Auth & validation filters
│   ├── models/           # Mongoose schemas
│   ├── routes/           # Express API routing tables
│   ├── server.js         # Server entry point
│   └── package.json
├── client/               # React Frontend (Vite)
│   ├── public/           # High-res optimized assets
│   ├── src/
│   │   ├── components/   # Reusable UI elements (Buttons, Cards)
│   │   ├── context/      # Global State (Booking/Menu)
│   │   ├── pages/        # 5 core routed pages
│   │   ├── styles/       # Tailwind CSS imports
│   │   ├── App.jsx       # App routes & layout root
│   │   └── main.jsx
│   └── package.json
```

2. Technical Architecture & Security Specifications

2.1 Backend Server Bootstrapping

The backend server utilizes Express.js to host REST endpoints and inject raw Socket.io listeners into the local HTTP server environment.

- **Database Client:** Mongoose ODM connects to MongoDB Atlas using connection pooling configured to preserve MO container memory:

```
mongoose.connect(process.env.MONGO_URI, {  
  maxPoolSize: 5, // Limit connections on free tier to prevent container overhead  
  serverSelectionTimeoutMS: 5000  
});
```

2.2 Security & Access Controls (API Level)

A demo site must be resilient to script injections and form spamming.

1. **Rate Limiting:** Protect the booking and inquiry endpoints from malicious bot scripts. We restrict traffic to a mathematical maximum R using `express-rate-limit : $$ R \leq \frac{100 \text{ booking requests}}{15 \text{ minutes}}` \$\$
2. **Input Sanitization:** Intercept all request body payloads to strip out SQL/NoSQL query selectors using `express-mongo-sanitize` and escape raw HTML tags using `xss-clean`.
3. **Authentication Control (Owner Console Access):**

- The Admin Panel endpoints (updating menu items/viewing active reservations) are strictly protected via **JSON Web Tokens (JWT)**.
- Tokens are generated upon successful login at `/api/auth/login` and sent inside an HTTP-only cookie to prevent Cross-Site Scripting (XSS) extraction.

3. Frontend Specification (React & Tailwind CSS)

3.1 Design Tokens & The Tailwind Configuration

The brand layout represents the hybrid nature of the culinary catalog: clean, minimalist Italian structuring combined with rich, ambient Arabic/Desi visuals (such as deep golds, warm charcoal, and vibrant spice tones).

- **Color Palette Token Definitions:**
 - Primary: Deep Amber/Gold (`#D4AF37`) — represents artisanal sweets and spices.
 - Secondary: Warm Charcoal/Onyx (`#1C1C1C`) — provides high-contrast fine-dining luxury backgrounds.
 - Neutral Accent: Cream/Soft White (`#FDFBF7`) — maintains layout readability.

3.2 Responsive CSS Layout Grid Strategy

Every layout dynamic component adheres to a flexible grid:

- On small viewports (width $< 640\text{px}$), columns collapse strictly to a single track (1fr).
- On medium-to-large viewports, grid columns auto-scale via CSS clamping rules:

```
grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
```

4. 10-Point Jira Feature Tickets (Day 2 - Day 7 Tasks)

These tickets are ready to be copied into your personal project manager or followed step-by-step.

Ticket 1: Initialize Project Directory & Environment

- **Priority:** High
- **Description:** Set up Client-Server directory structures, initialize npm environments, and construct the `.env` schema for both repositories.
- **Technical Checklist:**
 - ☐ Install Express, Mongoose, Dotenv, Cors, and Helmet on server repository.
 - ☐ Bootstrap the Client folder using Vite + React template.
 - ☐ Verify that Git ignores local server config secrets.
- **Acceptance Criteria:** Typing `npm run dev` in both directory levels executes the dev systems simultaneously with zero configuration warnings.

Ticket 2: Build MongoDB Connection Pipeline

- **Priority:** High
- **Description:** Formulate server connection modules using Mongoose and ensure failover safety during Atlas communication interruptions.
- **Technical Checklist:**
 - ☐ Connect to DB using environment variable credentials securely.
 - ☐ Log explicit warnings inside the CLI if the connection fails.
- **Acceptance Criteria:** Backend logs a success token: "MongoDB Atlas Connection Established Successfully."

Ticket 3: Develop Mongoose Models for Menu & Reservations

- **Priority:** High
- **Description:** Construct explicit schemas with indexing arrays to optimize query operations.
- **Technical Checklist:**

- ☐ Implement indexes on `menu.category` and `reservations.dateTime`.
- ☐ Implement strict Mongoose enumerations on statuses (`pending` , `confirmed`).
- **Acceptance Criteria:** Attempting to write an invalid reservation status errors out on mongoose pre-save validations.

Ticket 4: Build Express API Routes (CRUD for Menu & Booking)

- **Priority:** High
- **Description:** Write backend route handlers for user booking inputs and admin controls.
- **Technical Checklist:**
 - ☐ Map GET `/api/menu` to pull sorted lists from Atlas.
 - ☐ Map POST `/api/reservations` to validate fields via Joi/Yup.
- **Acceptance Criteria:** Submitting clean JSON to the booking router returns HTTP status code `201 Created`.

Ticket 5: Configure Socket.io Real-time Pipeline

- **Priority:** Medium
- **Description:** Inject Socket.io into the HTTP server wrapper. Broadcast active notifications on booking triggers.
- **Technical Checklist:**
 - ☐ Setup socket event emitters on booking completion routes.
 - ☐ Secure local socket origins via server CORS configuration.
- **Acceptance Criteria:** Instantiating a new reservation sends a backend-to-client payload containing the customer data within 500 milliseconds.

Ticket 6: Create React App Shell, Pages, & Navigation Routing

- **Priority:** High
- **Description:** Set up React Router, configure global styles, and build the persistent layout shell with navigation.

- **Technical Checklist:**
 - ☐ Install `react-router-dom` and define the 5 primary routes.
 - ☐ Build a responsive `<Navbar />` and a sticky mobile action tray.
- **Acceptance Criteria:** Moving between Navigation links transitions the browser smoothly without hard page reloads.

Ticket 7: Design the Hero Layer & Brand USP Showcase

- **Priority:** Medium
- **Description:** Code the visual entry page featuring your compressed background loop, brand messaging, and dual primary CTAs.
- **Technical Checklist:**
 - ☐ Style the layout using Tailwind's grid layout.
 - ☐ Bind the secondary CTA link to jump smoothly to the dynamic Menu category anchor.
- **Acceptance Criteria:** Clicking `[Book a Table]` instantly pops open a modal container or navigates straight to the booking route.

Ticket 8: Create the Tabbed Interactive Menu Component

- **Priority:** Medium
- **Description:** Build the responsive, data-driven Menu Grid. Categories should filter dynamically on client click.
- **Technical Checklist:**
 - ☐ Fetch the menu data on component mount.
 - ☐ Use local react state to filter visible cards based on the selected category tab.
- **Acceptance Criteria:** Clicking "Sweets & Desserts" filters and re-renders the visible menu cards instantaneously.

Ticket 9: Develop the Interactive Booking Form (Client Validation)

- **Priority:** High
- **Description:** Code the input forms with date selection restricts and client-side error helpers.
- **Technical Checklist:**
 - ☐ Disable booking dates prior to the current system date.
 - ☐ Implement loading indicators on form submission states.
- **Acceptance Criteria:** Submitting a booking displays a success card: "Table Secured for [Name] on [Date]!"

Ticket 10: Launch, Staging Setup, & SEO Analysis

- **Priority:** Medium
- **Description:** Connect the backend API to Render/Fly.io, host the client on Vercel/Netlify, and test performance.
- **Technical Checklist:**
 - ☐ Ensure client `.env.production` points to the hosted API server.
 - ☐ Set explicit image dimensions and alt text for search engine indexing.
- **Acceptance Criteria:** Running a Google Lighthouse performance analysis returns an audit score of ≥ 90 on both mobile and desktop.